Kigali Independent University (ULK)

UML Class Diagram Modelling Assignment

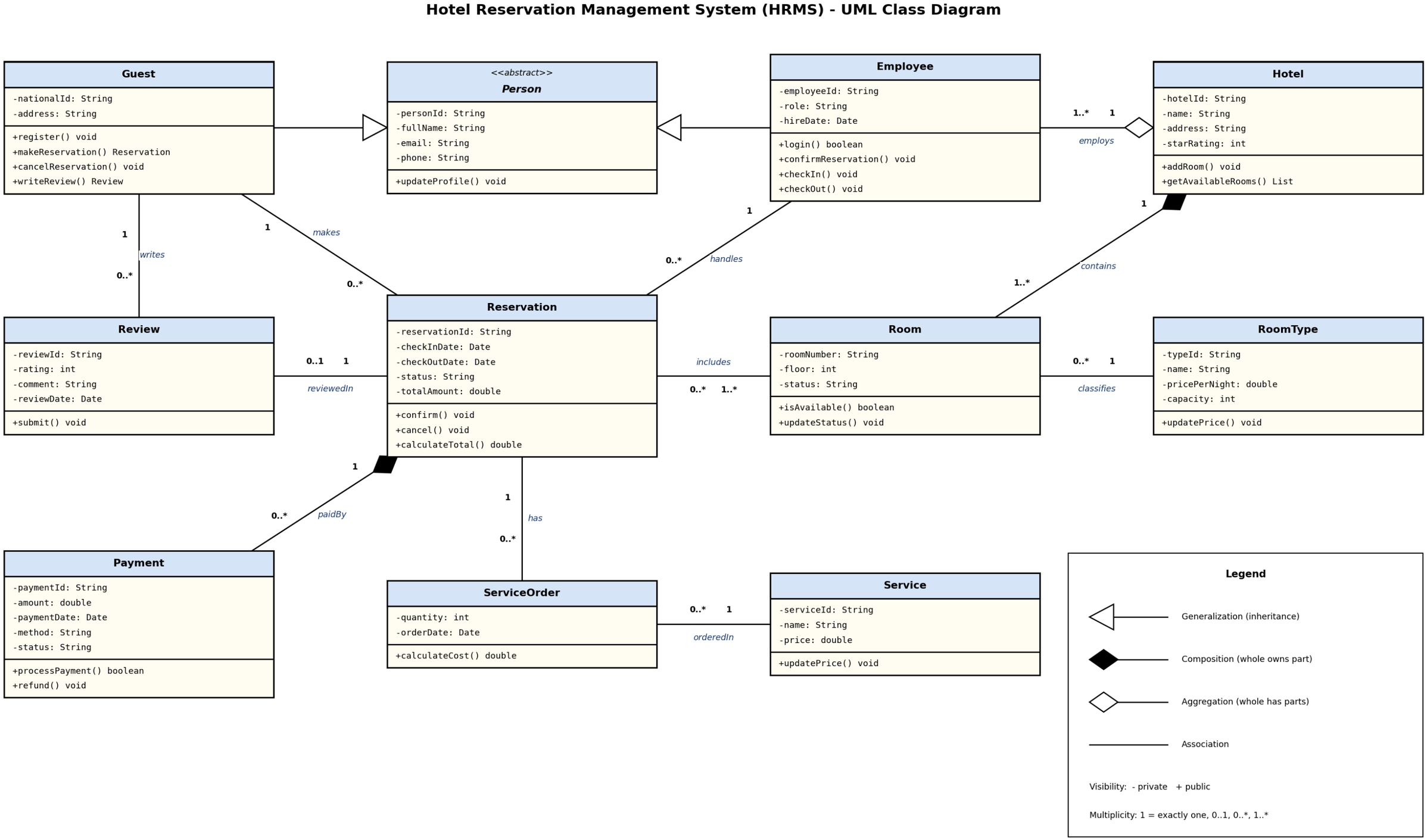
Report 2: UML Class Diagram Report

Hotel Reservation Management System (HRMS)

Project title	Hotel Reservation Management System (HRMS)
Instructor	Eric Maniraguha
Group member 1	Name: Ali Amin Student ID: 202512480
Group member 2	Name: Eihassan Ismail Student ID: 202513493
Release date	Thursday, 15 October 2026
Submission deadline	Tuesday, 20 October 2026, 8:30 AM

This report presents the complete UML class diagram of the HRMS, the classes with their attributes and methods, the relationships with their multiplicities, and a short explanation of each relationship.

1. Complete UML Class Diagram



2. Classes, Attributes, and Methods

All attributes are private (-) and all methods are public (+). Person is an abstract class and is shown in italics in the diagram.

Class	Attributes	Methods
Person (abstract)	personId: String fullName: String email: String phone: String	updateProfile(): void
Guest	nationalId: String address: String	register(): void makeReservation(): Reservation cancelReservation(): void writeReview(): Review
Employee	employeeId: String role: String hireDate: Date	login(): boolean confirmReservation(): void checkIn(): void checkOut(): void
Hotel	hotelId: String name: String address: String starRating: int	addRoom(): void getAvailableRooms(): List
RoomType	typeId: String name: String pricePerNight: double capacity: int	updatePrice(): void
Room	roomNumber: String floor: int status: String	isAvailable(): boolean updateStatus(): void
Reservation	reservationId: String checkInDate: Date checkOutDate: Date status: String totalAmount: double	confirm(): void cancel(): void calculateTotal(): double
Payment	paymentId: String amount: double paymentDate: Date method: String status: String	processPayment(): boolean refund(): void
Service	serviceId: String name: String price: double	updatePrice(): void
ServiceOrder	quantity: int orderDate: Date	calculateCost(): double
Review	reviewId: String rating: int comment: String reviewDate: Date	submit(): void

3. Relationships and Multiplicities

Relationship	Type	Multiplicity	Meaning
Guest, Employee to Person	Generalization	n/a	Guest and Employee both inherit the common attributes of Person.

Relationship	Type	Multiplicity	Meaning
Hotel to Room	Composition	1 to 1..*	A hotel contains one or more rooms; a room cannot exist without its hotel.
Hotel to Employee	Aggregation	1 to 1..*	A hotel employs one or more employees; employees remain independent objects.
RoomType to Room	Association	1 to 0..*	Each room has exactly one type; a type can classify many rooms.
Guest to Reservation	Association	1 to 0..*	A guest can make many reservations; each reservation belongs to one guest.
Employee to Reservation	Association	1 to 0..*	An employee handles many reservations; each reservation is handled by one employee.
Reservation to Room	Association	0..* to 1..*	A reservation includes one or more rooms; a room can appear in many reservations over time.
Reservation to Payment	Composition	1 to 0..*	Payments belong to exactly one reservation and have no meaning without it.
Reservation to ServiceOrder	Association	1 to 0..*	A reservation can have many service orders.
Service to ServiceOrder	Association	1 to 0..*	A service can appear in many service orders.
Guest to Review	Association	1 to 0..*	A guest can write many reviews.
Reservation to Review	Association	1 to 0..1	A reservation can have at most one review.

4. Explanation of the Relationships

Generalization. Guest and Employee are both people, so they inherit personId, fullName, email, phone, and updateProfile() from the abstract class Person. This avoids repeating the same attributes in two classes.

Composition (Hotel and Room, Reservation and Payment). These are strong whole-part relationships. If the hotel is removed from the system, its rooms are removed with it. If a reservation is deleted, its payments are deleted too. The filled diamond is drawn at the side of the whole.

Aggregation (Hotel and Employee). This is a weaker whole-part relationship. The hotel employs staff, but an employee is a person who continues to exist independently of one particular hotel record. The hollow diamond is drawn at the Hotel side.

Guest, Employee, and Reservation. A guest makes reservations (one guest, many reservations) and an employee handles them by confirming them and processing check-in and check-out.

Reservation and Room. One reservation can include several rooms (for example a family booking two rooms), and over time one room appears in many reservations. This many-to-many relationship is why both ends allow more than one object. The business rule that a room cannot be reserved for overlapping dates is enforced by Room.isAvailable() and Reservation.confirm().

RoomType and Room. Prices are stored once on the room type, so changing the price of a Suite updates it for every Suite room.

Reservation, ServiceOrder, and Service. A reservation can include many services and a service can be ordered in many reservations. This many-to-many relationship is resolved by the class ServiceOrder, which also stores the quantity and the order date. ServiceOrder therefore acts as an association class between Reservation and Service.

Reservation and Review, Guest and Review. After a stay, a guest can write reviews, and each reservation can have at most one review (multiplicity 0..1).